

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ
по лабораторной работе №2
по дисциплине «Логика и основы алгоритмизации в инженерных задачах»
на тему: «Оценка времени выполнения программ»

Выполнили ст.гр. 25ВВВ2:

Акимов М.А.

Трошин О.А.

Приняли:

к.т.н., доцент Юрова О.В.

к.т.н, ст. преподаватель, Деев М.В.

Пенза 2026

Цель работы: изучить методы оценки времени выполнения программ и сложности алгоритмов, сравнить теоретические оценки с экспериментальными данными.

Ход работы

Задание 1:

1. Вычислить порядок сложности программы (O-символику).

O-символика (или O-большое, записывается $O(\dots)$) показывает, как быстро растёт время работы программы, когда увеличивается размер входных данных. То есть она отвечает на вопрос: «Что произойдёт со временем работы программы, если размер задачи станет в 2, 10 или 100 раз больше?»

Например:

$O(n)$ — время растёт примерно пропорционально n ;

$O(n^2)$ — растёт как квадрат n ;

$O(n^3)$ — растёт как куб n (как и в нашем случае)

2. Оценить время выполнения программы и кода, выполняющего перемножение матриц, используя функции библиотеки `time.h` для матриц размерами от 100, 200, 400, 1000, 2000, 4000, 10000

Программа из задания доработана: перемножение вынесено в функцию с параметром n , добавлен сброс счётчика j (без него заполнялась только первая строка матрицы), матрицы объявлены глобально, иначе при $n = 10000$ они не помещаются на стеке, и добавлен замер времени. Порядок вложенных циклов изменён с $i-j-r$ на $i-r-j$ — в этом порядке элементы матриц перебираются по строкам, подряд, поэтому на больших размерах программа работает заметно быстрее. Текст программы 1 приведён ниже.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>
#include <windows.h>

#define MAXN 10000          // максимальный размер матрицы

/* Матрицы объявлены глобально: три массива 10000 x 10000
   занимают около 1,2 ГБ, на стеке столько места нет. */
int a[MAXN][MAXN], b[MAXN][MAXN], c[MAXN][MAXN];

void matrix_multiplication(int n)
{
    clock_t start, end;
    int i=0, j=0, r;
```

```

int elem_c;

srand((unsigned)time(NULL));
while(i<n)
{
    j=0;    // без сброса j заполнялась бы только первая
строка
    while(j<n)
    {
        a[i][j]=rand()% 100 + 1;    // заполняем матрицы
случайными числами
        b[i][j]=rand()% 100 + 1;
        c[i][j]=0;
        j++;
    }
    i++;
}

start = clock();

/* Порядок циклов i-r-j: элементы b и c перебираются
по строкам, подряд. При обычном порядке i-j-r матрица b
читается по столбцам, и программа работает дольше. */
for(i=0;i<n;i++)
{
    for(r=0;r<n;r++)
    {
        elem_c=a[i][r];
        for(j=0;j<n;j++)
            c[i][j]=c[i][j]+elem_c*b[r][j];
    }
}

end = clock();

printf("%d\t%f сек.\n", n,
(double) (end-start)/CLOCKS_PER_SEC);
}

int main(void)
{
    int sizes[] = { 100, 200, 400, 1000, 2000, 4000, 10000 };
    int count = sizeof(sizes) / sizeof(sizes[0]);
    int k;

    setvbuf(stdin, NULL, _IONBF, 0);
    setvbuf(stdout, NULL, _IONBF, 0);
    SetConsoleOutputCP(1251);    // вывод по-русски
    setlocale(LC_NUMERIC, "Russian");    // запятая в дробях

    printf("Размер\tВремя\n");

    for(k=0; k<count; k++)

```

```

        matrix_multiplication(sizes[k]);

    return 0;
}

```

Результат программы 1 представлен на рисунке 1.

Размер	Время
100	0,000000 сек.
200	0,001000 сек.
400	0,007000 сек.
1000	0,152000 сек.
2000	2,717000 сек.
4000	24,101000 сек.
10000	317,852000 сек.

Рис.1 – результат программы 1 к заданию 1.

3. Построить график зависимости времени выполнения программы от размера матриц и сравнить полученный результат с теоретической оценкой.

График зависимости времени выполнения программы от размера матриц представлен на рисунке 2.

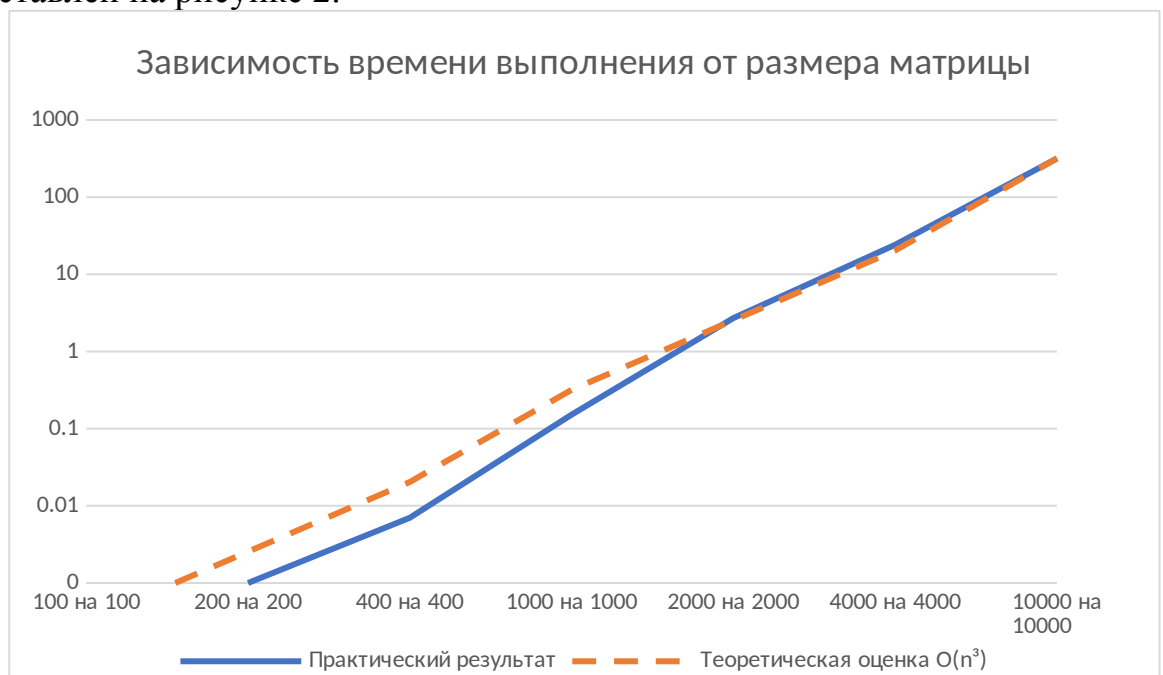


Рис.2 – график зависимости времени выполнения программы от размера матриц.

Дана реализация алгоритмов сортировки Шелла и быстрой сортировки:

```

void shell(int *items, int count)
{

```

```

int i, j, gap, k;
int x, a[5];

a[0]=9; a[1]=5; a[2]=3; a[3]=2; a[4]=1;

for(k=0; k < 5; k++) {
    gap = a[k];
    for(i=gap; i < count; ++i) {
        x = items[i];
        for(j=i-gap; (x < items[j]) && (j >= 0); j=j-gap)
            items[j+gap] = items[j];
        items[j+gap] = x;
    }
}

void qs(int *items, int left, int right) // вызов функции: q
s(items, 0, count-1);
{
    int i, j ;
    int x, y;

    i = left; j = right;
    /* выбор компаранда */
    x = items[(left+right)/2];

    do {
        while((items[i] < x) && (i < right)) i++;
        while((x < items[j]) && (j > left)) j--;

        if(i <= j) {
            y = items[i];
            items[i] = items[j];
            items[j] = y;
            i++; j--;
        }
    } while(i <= j);

    if(left < j) qs(items, left, j);
    if(i < right) qs(items, i, right);
}

```

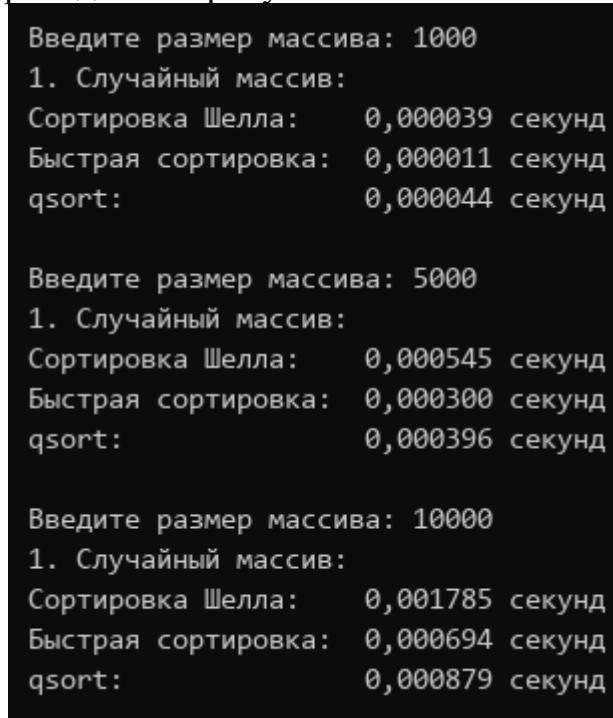
Задание 2:

Функции `shell` и `qs` взяты из задания без изменений: в сортировке Шелла переставлены местами условия внутреннего цикла, так как в исходном виде элемент `items[j]` читается до проверки индекса, то есть за границей массива. Вокруг них написана программа, которая запрашивает размер массива, формирует четыре набора данных (случайный, возрастающий, убывающий и «половина возрастает, половина убывает») и замеряет на каждом из них время работы обеих сортировок и стандартной функции `qsort`. Одна

сортировка небольшого массива выполняется быстрее, чем тикает таймер, поэтому каждый алгоритм запускается в цикле, пока не наберётся 0,2 секунды, и выводится среднее время одной сортировки. Замеры выполнены для массивов из 1000, 5000, 10000 элементов.

1. Оценить время работы каждого из реализованных алгоритмов на случайном наборе значений массива.

Время работы алгоритмов на случайном наборе значений для всех трёх размеров массива приведено на рисунке 3.



Введите размер массива:	1. Случайный массив:
1000	
Сортировка Шелла:	0,000039 секунд
Быстрая сортировка:	0,000011 секунд
qsort:	0,000044 секунд
5000	
Сортировка Шелла:	0,000545 секунд
Быстрая сортировка:	0,000300 секунд
qsort:	0,000396 секунд
10000	
Сортировка Шелла:	0,001785 секунд
Быстрая сортировка:	0,000694 секунд
qsort:	0,000879 секунд

Рис.3 – время сортировок на случайном массиве.

2. Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой возрастающую последовательность чисел.

Время работы на возрастающей последовательности приведено на рисунке 4.

```

Введите размер массива: 1000
2. Возрастающий массив:
Сортировка Шелла:      0,000005 секунд
Быстрая сортировка:    0,000006 секунд
qsort:                  0,000017 секунд

Введите размер массива: 5000
2. Возрастающий массив:
Сортировка Шелла:      0,000024 секунд
Быстрая сортировка:    0,000034 секунд
qsort:                  0,000099 секунд

Введите размер массива: 10000
2. Возрастающий массив:
Сортировка Шелла:      0,000047 секунд
Быстрая сортировка:    0,000074 секунд
qsort:                  0,000211 секунд

```

Рис.4 – время сортировок на возрастающем массиве.

3. Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой убывающую последовательность чисел.

Время работы на убывающей последовательности приведено на рисунке 5.

```

Введите размер массива: 1000
3. Убывающий массив:
Сортировка Шелла:      0,000039 секунд
Быстрая сортировка:    0,000006 секунд
qsort:                  0,000019 секунд

Введите размер массива: 5000
3. Убывающий массив:
Сортировка Шелла:      0,000726 секунд
Быстрая сортировка:    0,000035 секунд
qsort:                  0,000107 секунд

Введите размер массива: 10000
3. Убывающий массив:
Сортировка Шелла:      0,002812 секунд
Быстрая сортировка:    0,000074 секунд
qsort:                  0,000223 секунд

```

Рис.5 – время сортировок на убывающем массиве.

4. Оценить время работы каждого из реализованных алгоритмов на массиве, одна половина которого представляет собой возрастающую последовательность чисел, а вторая, – убывающую.

Время работы на массиве, одна половина которого возрастает, а вторая убывает, приведено на рисунке 6.

```
Введите размер массива: 1000
4. Половина возрастающая, половина убывающая:
Сортировка Шелла:      0,000022 секунд
Быстрая сортировка:    0,000112 секунд
qsort:                  0,000048 секунд

Введите размер массива: 5000
4. Половина возрастающая, половина убывающая:
Сортировка Шелла:      0,000398 секунд
Быстрая сортировка:    0,002308 секунд
qsort:                  0,000375 секунд

Введите размер массива: 10000
4. Половина возрастающая, половина убывающая:
Сортировка Шелла:      0,001449 секунд
Быстрая сортировка:    0,009242 секунд
qsort:                  0,000842 секунд
```

Рис.6 – время сортировок на массиве «половина возрастает, половина убывает».

5. Оценить время работы стандартной функции qsort, реализующей алгоритм быстрой сортировки на выше указанных наборах данных.

Время работы стандартной функции qsort на всех четырёх наборах данных приведено на рисунке 7, сводные значения по всем размерам массива — в таблице.

```
Введите размер массива: 10000

1. Случайный массив:
qsort:                  0,000879 секунд

2. Возрастающий массив:
qsort:                  0,000211 секунд

3. Убывающий массив:
qsort:                  0,000223 секунд

4. Половина возрастающая, половина убывающая:
qsort:                  0,000842 секунд
```

Рис.7 – время стандартной функции qsort (массив из 10000 элементов).

Алгоритм	Случайный (сек)	Возрастающий(сек)	Убывающий (сек)	Пик (сек)
shell, n = 1000	0,000039	0,000005	0,000039	0,000022
shell, n = 5000	0,000545	0,000024	0,000726	0,000398
shell, n = 10000	0,001785	0,000047	0,002812	0,001449
qs, n = 1000	0,000011	0,000006	0,000006	0,000112
qs, n = 5000	0,000300	0,000034	0,000035	0,002308
qs, n = 10000	0,000694	0,000074	0,000074	0,009242
qsort, n = 1000	0,000044	0,000017	0,000019	0,000048
qsort, n = 5000	0,000396	0,000099	0,000107	0,000375
qsort, n = 10000	0,000879	0,000211	0,000223	0,000842

Вывод: экспериментально подтверждена кубическая сложность умножения матриц — $O(n^3)$: при увеличении размера матрицы в 2,5 раза время растёт примерно в 13,2 раза (теоретическая оценка — в 15,6 раза), а отклонения на средних размерах объясняются тем, что матрицы перестают помещаться в кэш процессора. Сравнение сортировок выполнено на массивах из 1000, 5000, 10000 элементов. На случайных и убывающих данных быстрая сортировка заметно опережает сортировку Шелла: на массиве из 10000 элементов 0,000694 с против 0,001785 с. Однако на массиве «половина возрастает, половина убывает» выбор среднего элемента в качестве компаранда приводит к вырождению быстрой сортировки до $O(n^2)$: её время растёт с 0,000112 с при 1000 элементах до 0,009242 с при 10000 — примерно в 82,5 раза при увеличении размера массива в 10 раз, что и соответствует квадратичной зависимости. Стандартная функция qsort показала стабильное время на всех наборах данных. Таким образом, выбор алгоритма сортировки зависит от характера входных данных.